INTRODUCTION TO NEURAL NETWORK AND THE IMPLEMENTATION

HYUN JUN YOO

Abstract. We investigate fully connected deep neural networks for image classification on FashionMNIST dataset. A baseline FCN with ReLU activation is first established, then evaluated across three dimensions: optimizer choice (SGD, RMSprop, Adam) at varying learning rates, weight initialization strategies (Random Normal, Xavier, Kaiming Uniform), and the effect of batch normalization. Result show that Adam with learning rate 0.001 achieves the highest test accuracy of 89.57%, while batch normalization slightly improved performance. These findings highlight the practical importance of hyperparameter tuning in training of deep networks.

1. Introduction and Overview

This project explores the implementation and optimization of Fully Connected Deep Neural Networks for the task of image classification. The primary focus is the FashionMNIST dataset, which consists of grayscale images categorized into 10 distinct classes of clothing. Each image is a 28×28 pixel grid, which must be flattened into a 784-dimensional vector for processing by the network.

The goal of this report is to establish a baseline model using the ReLU activation function and SGD optimizer that achieves a minimum of 85% testing accuracy within reasonable training time, and from the baseline model, conduct hyperparameter tuning to systematically evaluate the effects of different optimizers (SGD, RMSprop, Adam), weight initialization (Random Normal, Xavier, Kaiming Uniform), and the inclusion of Batch Normalization on model performance.

By analyzing loss curves and accuracy metrics across these variables, this report seeks to determine the most effective means to train deep neural networks on complex image data.

2. Theoretical Background

A Fully Connected Network consists of an input layer $a^{(0)}$, ℓ hidden layers, and an output layer $a^{\ell+1}$. Each layer performs a linear transformation followed by a nonlinear activation function σ. The pre-activation z^l and the activation a^l are defined as:

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}$$
$$a^{(l)} = \sigma(z^{(l)})$$

where W and b are weight matrix and bias vector. To enable the network to learn complex, non-linear patterns, a non-linearity must be introduced. This project employs the ReLU ($\sigma(z) = \max(0, z)$). ReLU is computationally efficient and helps mitigate the vanishing gradient problem by maintaining a unit gradient for all positive inputs, ensuring that the learning signal remains significant enough across structure.

Primary challenge in training deep networks is the instability of gradients during backpropagation. The gradient of the cost function with respect to the weights of the early layers involves a product of weight matrices and derivatives of activation functions from all subsequent layers. These gradients can grow exponentially or diminish exponentially. These phenomenons are called exploding/vanishing gradient. Exploding gradient leads to numerical instability, and vanishing gradient prevents the early layers from updating effectively, stalling the learning process. These issues are adressed by normalizing input data, precise choice of parameter initialization, and batch normalization. To train the network, we use gradient descent.

$$w^{(l)} \leftarrow w^{(l)} - n \frac{\partial L}{\partial W^{(l)}}$$
$$b^{(l)} \leftarrow b^{(l)} - n \frac{\partial L}{\partial b^{(l)}}$$

Date: April 13, 2026.

We compute all these derivatives with backpropagation algorithm. Backpropagation is an application of the chain rule, storing intermediate quantities so that they can be reused. Backpropagation allows the model to adjust internal representations by propagating the final classification error backward through the network.
While standard Stochastic Gradient Descent updates parameters using a fixed fraction of the gradient, it can suffer from slow convergence in complex settings. To improve efficiency, we look into two more optimizers.
First is RMSProp. This method normalizes the gradient by a running average of its recent magnitudes, preventing the learning rate from diminishing too quickly.
Second is Adam. This optimizer uses both the advantages of momentum and RMSprop techniques to adjust learning rates during training. It is known to work well with large datasets and complex model, thanks to its memory efficiency.
The convergence of model is highly sensitive to initial distribution of weights. If initial values are poorly scaled, signal variance can either explode or vanish as it propagates through the layers. Three strategies are evaluated to ensure numerical stability.
Random normal initialization samples weights from a Gaussian distribution with a fixed mean and variance. While simple, this method can lead to exploding or vanshing gradients, because it doesn't account for the number of input connections to each neuron.
We compared Random Normal, Xavier, and Kaiming initialization strategies to maintain signal variance across layers . While Xavier is designed for symmetric activations, Kaiming initialization specifically compensates for ReLU's half-unit deactivation by doubling the required weight variance to $2/fan_in$

3. Algorithm Implementation and Development

This project used various libraries. NumPy [1] was used for mathematical operations and data manipulation, scikit-learn [2] was used for data partitioning, PyTorch [3] was used as deep learning framework for tensor operations and neural network construction, Torchvision [4] for efficient loading of data and transformation of the said data, and Matplotlib [5] to generate the loss curves and accuracy plots.
The dataset is accessed and preprocessed using specialized computer vision utilities. Each 28×28 image is converted into a tensor and normalized using a mean of 0.2860 and standard deviation of 0.3530. This ensures that the input features occupy a similar numerical range, which prevents specific pixels from disproportionately affecting the gradient update.
To ensure the model generalizes well to unseen data, the training set is partitioned into two subsets, where one of them serves as a training set for parameter optimization and a validation set for hyperparameter tuning. We take this approach instead of cross-validation as neural network is computationally expensive. A stratified split is employed to maintain the relative proportions of the ten classes across both sets, ensuring that the validation metrics are a reliable metric for model performance.
The project adapted modular architecture design to allow easier navigation of different options. Network is implemented as a flexible class structure capable of supporting varying depths and widths. To avoid rigid architectural constraints, a dynamic container is used to store a sequence of linear transformations and activation functions. Such modular approach allows various choice of options such as optimizers, initialization, and use of batch normalization to be passed as arguments.

4. Computational Results

The baseline configuration was defined as a network with 2 hidden layers with 512 hidden neurons each. The model is trained over 40 epochs, using the SGD optimizer with a learning rate of 0.01 and Kaiming initialization, which is the default configuration by PyTorch convention. This baseline configuration achieved 86.73%. The extended training of 40 epochs allowed the baseline model to move past the 85% accuracy. With lower learning rate of lr = 0.001, performance of model degraded. The test accuracy resulted in 81.93%.

We move onto testing Random Normal. Huge drop in first epoch in figure 2 shows that the model is experiencing instability, and validation accuracy fluctuated around 70%. As these metrics show, the testing accuracy degraded heavily as expected, being just 70.58%.
An interesting feature of Random Normal initialization is that with SGD, the model performs better with

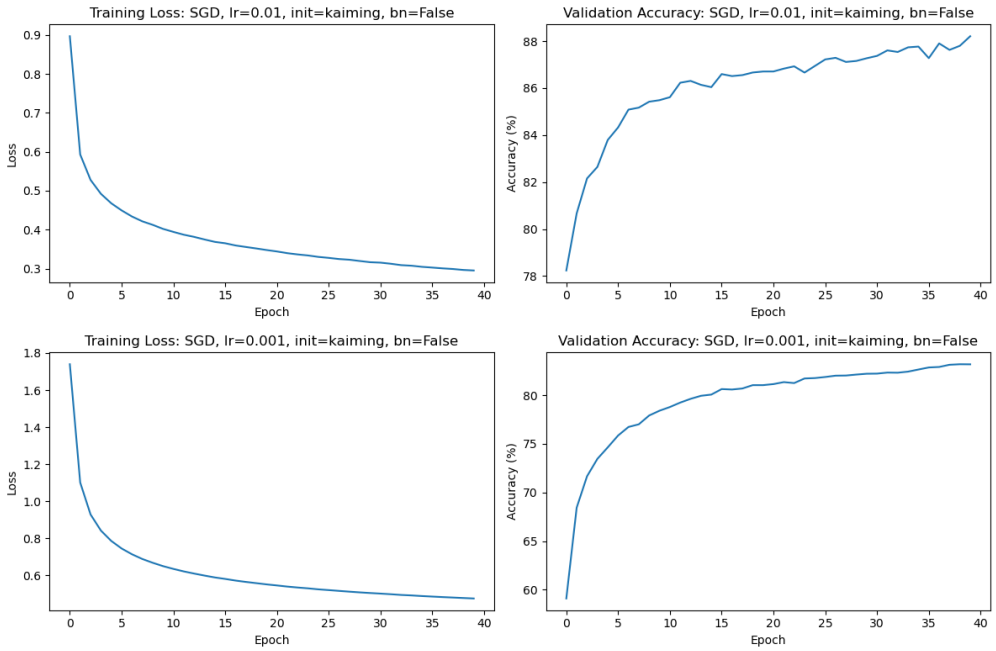

FIGURE 1. Performance of Baseline Model

learning rate of 0.001, even though the default configuration of PyTorch uses learning rate of 0.01 for SGD. This is expected from the training loss plot, as it shows a huge descent on first epoch.

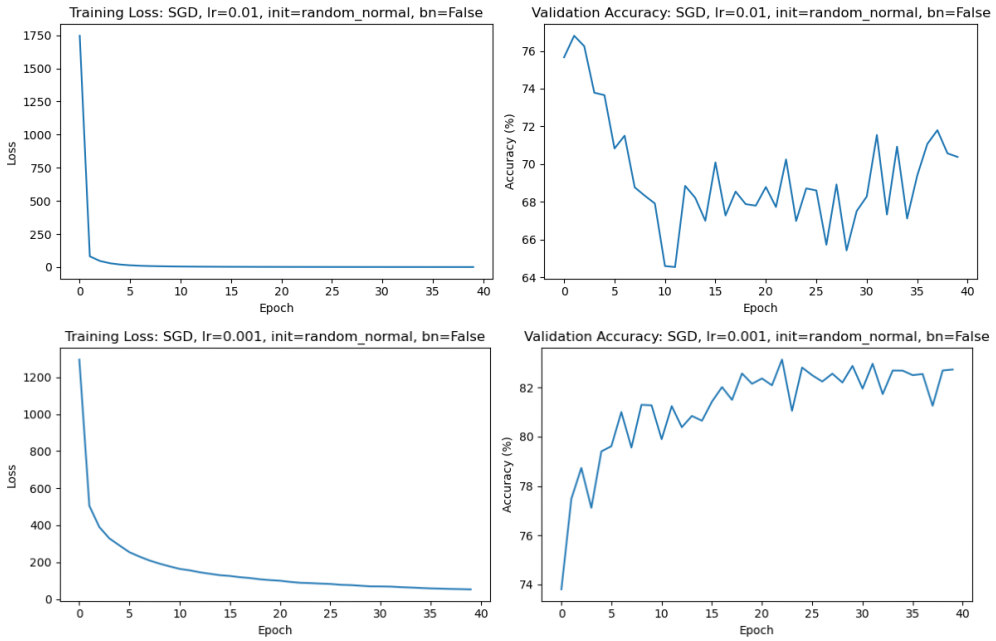

FIGURE 2. Random Normal Initialization with SGD

Test accuracy for was 81.57%. While better than previous model, this still showed worse performance than our baseline model.

We move onto Xavier Normal initialization. While performing better than Random Normal, Xavier Normal proved to be slightly worse than our baseline model. Our attempts resulted in test accuracy of 86.69% and 81.53% for learning rate of 0.01 and 0.001 each.

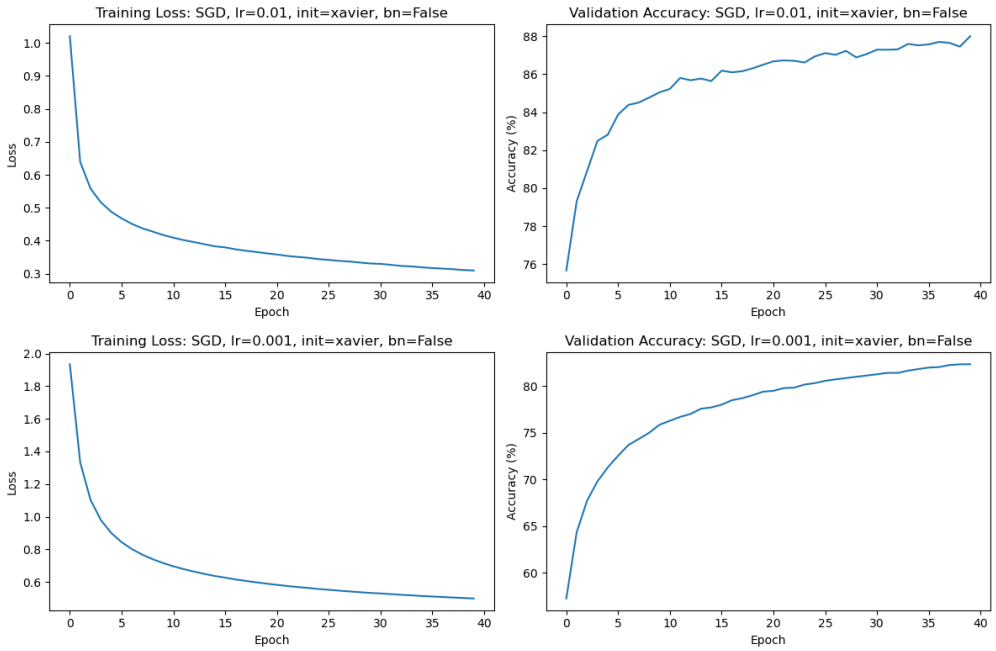

FIGURE 3. Xavier Normal with SGD

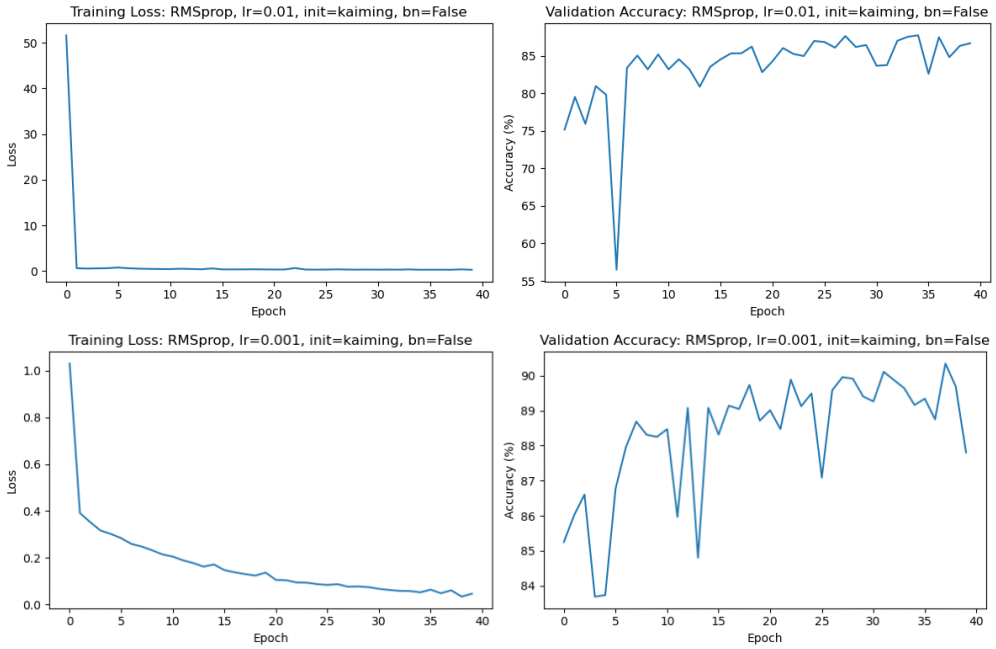

FIGURE 4. RMSProp

Similar to initializations, we compared the performance of RMSprop and Adam across two learning rates of 0.01 and 0.001. RMSprop plotted by figure 4 showed improved performance from our baseline model. While validation accuracy fluctuated, test accuracy resulted 86.01% and 88.96%, which is marginally higher than our baseline model.

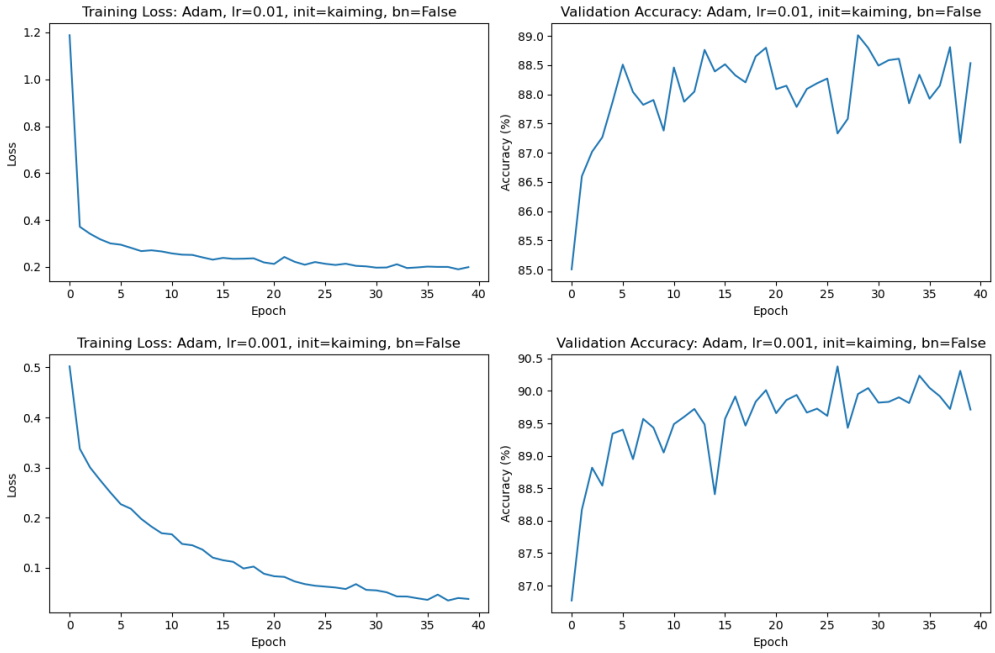

FIGURE 5. Adam

Adam plotted by figure 5 showed even higher test accuracy of 87.47% and 89.57% for learning rate of 0.01 and 0.001 each. It is worth noting that while SGD performed better with learning rate of 0.01, performance of Adam and RMSprop improved at learning rate of 0.001.

Lastly, we take a look at how Batch Normalization affects the performance of our model. Batch Normalization showed little improvement to test accuracy when paired with SGD. The test accuracy was 86.92%, which is higher than our baseline model's test accuracy, which was 86.73%. It shows a direct tradeoff where Batch Normalization provides slightly improved accuracy, while it makes computation take slightly more time. The training loss and validation accuracy is shown in figure 6.

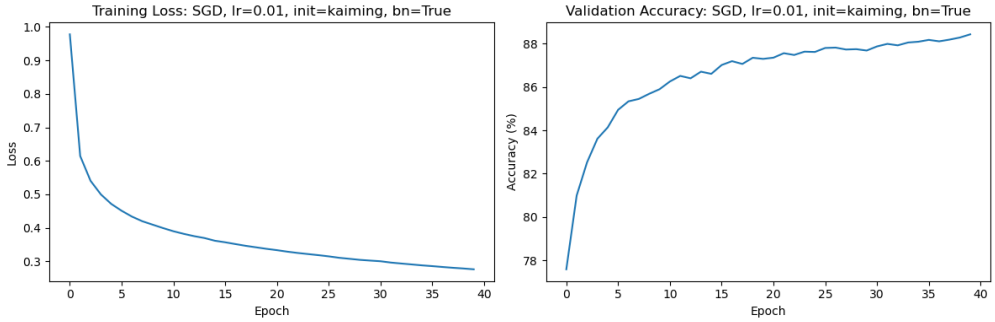

FIGURE 6. Baseline Model with Batch Normalization

5. Summary and Conclusions

This project successfully established a modular framework for classifying the FashionMNIST dataset using Fully Connected Networks. By developing a baseline model with two hidden layers and 512 neurons, we demonstrated that a relatively simple architecture can achieve robust classification accuracy when trained over 40 epochs. The subsequent hyperparameter exploration revealed that while the baseline configuration met the 85% performance threshold, the efficiency and stability of the network were significantly enhanced by moving beyond standard Stochastic Gradient Descent.

The most significant discovery was the clear synergy between adaptive optimization and informed weight initialization. Adam, specifically at a learning rate of 0.001, outperformed SGD by a large margin. Furthermore, the transition from Random Normal to Kaiming initialization proved essential for maintaining gradient flow through the wide hidden layers, as Kaiming specifically accounts for the ReLU activation function. The integration of Batch Normalization also proved to be beneficial for the accuracy of model at the cost of computational expense.

To conclude, Kaiming initialization, Adam optimizer, and introduction of Batch Normalization proved to be beneficial for the accuracy of model.

References

[1] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, September 2020.

[2] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[3] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. *PyTorch: an imperative style, high-performance deep learning library*. Curran Associates Inc., Red Hook, NY, USA, 2019.

[4] Sébastien Marcel and Yann Rodriguez. Torchvision the machine-vision package of torch. In *Proceedings of the 18th ACM International Conference on Multimedia*, MM '10, page 1485–1488, New York, NY, USA, 2010. Association for Computing Machinery.

[5] J. D. Hunter. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007.